TP01: Indexação Geoespacial e Geometria Computacional Híbrida

Larissa Oliveira Aguiar

21 de outubro de 2026

O código-fonte completo e os arquivos do projeto estão disponíveis no repositório do GitHub: `https://github.com/larisaguiar/Indexao-Geoespacial`.

1 Introdução

O trabalho prático propõe a implementação de um serviço de geolocalização por meio de técnicas e algoritmos de Geometria Computacional. Nesse trabalho, foram usadas estrutura de dados como árvores de prefixos (Tries) aplicadas à codificação Geohash para explorar a busca de dados e informações geográficos. Nesse sentido, foi possível relacionar além do conteúdo teórico, a aplicabilidade de algoritmos para o desenvolvimento de, por exemplo, sistemas de rastreamento em tempo real.

2 Estrutura de Dados

2.1 GeoHash

Geohashing é um método de geocodificação baseado em hash que codifica coordenadas geográficas, como latitude e longitude, em sequências alfanuméricas. O algoritmo Geohash divide o espaço em uma grade com 32 células. Usando a codificação alfabética Base-32, o primeiro caractere do Geohash identifica a localização inicial como uma das 32 células. A grade é dividida recursivamente em células menores.

2.2 Árvore de Prefixos

Para armazenar e recuperar de forma eficiente os códigos Geohash gerados para a frota de entregadores em operação em Belo Horizonte, utilizou-se uma estrutura de Árvore Trie Compacta , conforme as restrições do trabalho. A escolha da Trie é devido a natureza hierárquica dos códigos Geohash, como prefixos comuns representam regiões geográficas vizinhas e cada caractere adicional refina a precisão espacial, a árvore permite buscas por prefixo mais rápidas.

2.3 Busca Geoespacial

Cada entregador possui latitude e longitude. Essas coordenadas são codificadas em uma string Geohash Base32 com precisão de ate 8 caracteres. Os Geohashes são inseridos em uma Radix Tree; códigos iguais compartilham a mesma entrada no índice e ficam associados aos respectivos entregadores.

Ao fazer uma busca, o sistema geocodifica o estabelecimento escolhido, calcula um prefixo Geohash adequado ao raio e consulta a célula central e as celulas adjacentes. A consulta das vizinhas evita descartar entregadores próximos que estejam do outro lado de uma borda da grade.

A construção do índice percorre a frota. Com o índice pronto, a busca por prefixo e a filtragem geométrica dependem do tamanho dos prefixos consultados e do numero de candidatos examinados. Temos, O(m + k), onde m e o tamanho do prefixo e k e o numero de candidatos.

2.4 Distância

A fórmula de Haversine é um método para calcular a distância de círculo máximo entre dois pontos sobre uma esfera, a partir de suas latitudes e longitudes. O círculo máximo representa o menor caminho sobre a superfície esférica entre os pontos. Em aplicações geográficas, essa distância é uma aproximação da distância sobre a superfície da Terra, ela não corresponde necessariamente ao trajeto que uma pessoa ou veículo percorreria.

Para os pontos $(\text{latitude}_1, \text{longitude}_1)$ e $(\text{latitude}_2, \text{longitude}_2)$, primeiro convertem-se as coordenadas de graus para radianos. Definem-se as variações: $\Delta_{\text{lat}} = \text{latitude}_2 - \text{latitude}_1$ e $\Delta_{\text{lon}} = \text{longitude}_2 - \text{longitude}_1$.

O cálculo principal é dado por:

$$a = \sin\left(\frac{\Delta_{\text{lat}}}{2}\right)^2 + \cos(\text{latitude}_1) \cdot \cos(\text{latitude}_2) \cdot \sin\left(\frac{\Delta_{\text{lon}}}{2}\right)^2$$

$$c = 2 \cdot \text{atan2}\left(\sqrt{a}, \sqrt{1-a}\right)$$

$$\text{distância} = R \cdot c$$

As latitudes e longitudes nas expressões trigonométricas devem estar em radianos. R é o raio médio adotado para a Terra na unidade desejada. Neste trabalho, a função `calcular_haversine` usa $R = 6371$ km por padrão, mas também aceita metros ($R = 6371000$) e milhas ($R = 3958.8$). O ângulo c é medido em radianos, portanto, multiplicar c por R produz uma distância na mesma unidade do raio.

No sistema, a fórmula é aplicada após a busca por prefixo na Radix Tree. O Geohash e a Trie reduzem o conjunto da frota a candidatos em células próximas. O Haversine calcula a distância exata entre cada candidato e as coordenadas do estabelecimento. O candidato é considerado dentro da área somente quando a distância calculada é menor ou igual ao raio selecionado pelo usuário. Essa etapa geométrica final também descarta candidatos que entraram na busca por estarem em uma célula adjacente, mas que na realidade estão fora do raio. A distância é em linha reta sobre a superfície, não considerando ruas, obstáculos, relevo ou trânsito.

3 Serviço

3.1 Visão Geral

O projeto e um página para localizar entregadores em Belo Horizonte. O usuário seleciona um estabelecimento do catalogo, define um raio de busca e consulta quais entregadores disponíveis estão próximos. O painel mostra os resultados no mapa e em uma lista com distancia e tempo estimado. Os entregadores são dados fictícios gerados pelo sistema. Os estabelecimentos são lidos do arquivo 'bares.csv' que foi disponibilizado .

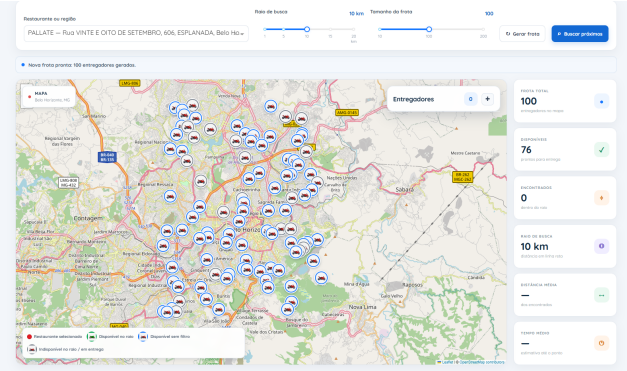


Figura 1: Visão Geral da Interface

3.2 Funcionalidades

As principais funcionalidades implementadas incluem:

- **Pesquisa de Estabelecimentos:** Busca flexível por nome, endereço ou bairro diretamente na base de dados de bares.
- **Geocodificação:** Utilização da API do Nominatim (OpenStreetMap) para conversão de endereços em coordenadas geográficas.
- **Localização Aproximada:** Mecanismo de fallback que utiliza o bairro como referência espacial caso o endereço exato não seja localizado.
- **Geração de Frota:** Criação parametrizada de entregadores fictícios com distribuição espacial configurável em Belo Horizonte.
- **Busca Geoespacial por Raio:** Filtragem de entregadores utilizando a Radix Tree combinada com a fórmula de Haversine.
- **Visualização Cartográfica:** Exibição interativa no mapa destacando o estabelecimento selecionado e os entregadores disponíveis.
- **Painel de Resultados:** Listagem detalhada dos entregadores encontrados, contendo a distância calculada em quilômetros e o tempo estimado de deslocamento.
- **Indicadores de Desempenho da Frota:** Métricas exibindo o total de entregadores cadastrados, a quantidade disponível e o subconjunto filtrado pela busca.

3.3 Arquitetura e Organização dos Arquivos

A tabela a seguir resume a estrutura modular do projeto, detalhando a função de cada arquivo no trabalho:

Tabela 1: Estrutura de arquivos e módulos do sistema

Arquivo	**Descrição e Responsabilidade**
app.py	Ponto de entrada; cria a aplicação Dash, monta o layout e gerencia as interações.
src/fleet.py	Gera a frota de entregadores, constrói o índice espacial e filtra os candidatos.
src/geohash.py	Codifica/decodifica Geohash e implementa o cálculo de distâncias via Haversine.
src/trie.py	Implementa a estrutura Radix Tree (Trie compacta) para indexação eficiente.
src/restaurants.py	Carrega o catálogo, consulta a API do Nominatim e gerencia o fallback por bairro.
src/components.py	Cria os elementos do mapa, marcadores personalizados, legendas e painéis.
src/interface.py	Define os estilos base e componentes visuais da interface em Python.
src/bares.csv	Base de dados local contendo o catálogo de estabelecimentos.
assets/dashboard.css	Estilos para layout, mapa e responsividade.
assets/moto-vermelha.jpg	Ícone gráfico utilizado nos marcadores dos entregadores.
requirements.txt	Especifica as dependências e bibliotecas externas do Python.
Dockerfile	Configuração do Docker para empacotamento e execução da aplicação.

4 Tecnologias e Dependências

O projeto foi desenvolvido utilizando a linguagem Python e bibliotecas especializadas para manipulação de dados, computação numérica, renderização cartográfica e construção da interface. A tabela a seguir lista as principais dependências do sistema:

Tabela 2: Principais tecnologias e dependências do projeto

Tecnologia / Biblioteca	**Descrição e Utilização no Projeto**
Python 3.10+	Linguagem de programação utilizada no desenvolvimento.
Dash	Framework web para construção da aplicação interativa, gerenciando o layout e as reações (*callbacks*).
dash-leaflet	Componentes interativos de mapa baseados em Leaflet integrados à interface Dash.
NumPy	Biblioteca de computação científica para operações numéricas e cálculos .
pandas	Leitura, tratamento e manipulação do catálogo de estabelecimentos em formato CSV.
Bibliotecas Padrão	Módulos nativos para requisições HTTP, JSON, funções matemáticas, controle de tempo e estruturas auxiliares.

5 Instruções de Execução

O projeto pode ser executado diretamente no ambiente local utilizando um ambiente virtual Python ou de forma isolada através de um contêiner Docker.

5.1 Execução Local

Na raiz do projeto, crie e ative o ambiente virtual e instale as dependências executando os seguintes comandos no terminal:

```
python -m venv .venv
# Ative o ambiente virtual de acordo com  sistema operacional
python -m pip install -r requirements.txt
python app.py
```

5.2 Execução com Docker

Caso prefira utilizar o Docker para empacotamento e isolamento, execute os comandos abaixo na raiz do projeto para construir a imagem e iniciar o contêiner:

```
docker build -t tp01 .
docker run --rm -p 8050:8050 tp01
```

Se a porta 8050 já estiver em uso, pode mapeá-la para outra porta local, por exemplo:

```
docker run --rm -p 8051:8050 tp01
```

6 Seção Experimental

6.1 Testes Práticos e Cenários de Validação

Para validar o funcionamento integrado do sistema, foram realizados testes práticos variando parâmetros operacionais, como o raio de abrangência e a quantidade da frota simulada.

6.1.1 Teste 1: Restaurante Pallate

Neste primeiro cenário de validação, foram definidos os seguintes parâmetros de execução:

- **Estabelecimento de Referência:** Restaurante Pallate.
- **Raio de Busca:** 10 km.
- **Tamanho da Frota:** 100 entregadores gerados aleatoriamente

A Figura 3 demonstra o resultado parcial da geração da frota e da filtragem espacial sob esses parâmetros:

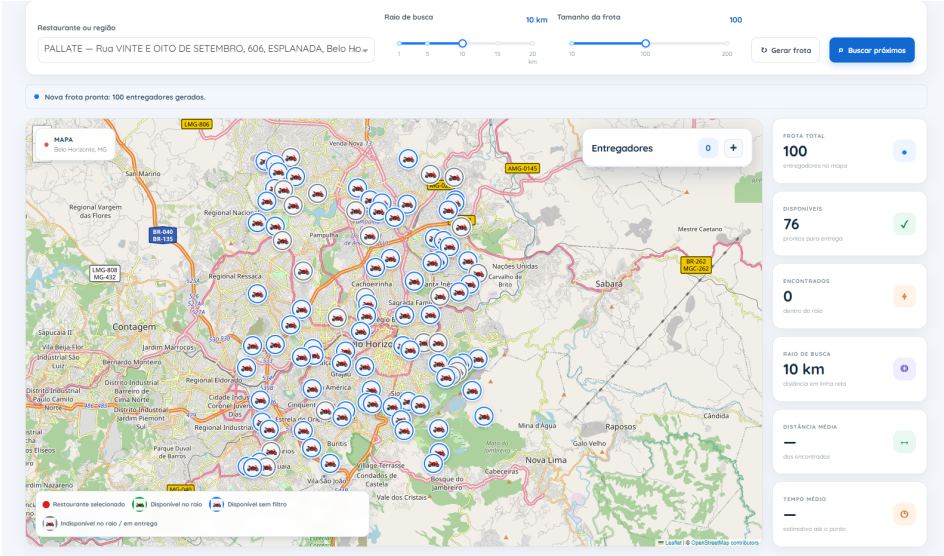


Figura 2: Resultado do Teste 1: Visualização da frota e busca para o Restaurante Pallate.

A Figura 3 demonstra o resultado da execução com os parâmetros acima, destacando a visualização cartográfica no mapa interativo, a listagem de entregadores disponíveis por proximidade no painel lateral e as métricas gerais da frota.

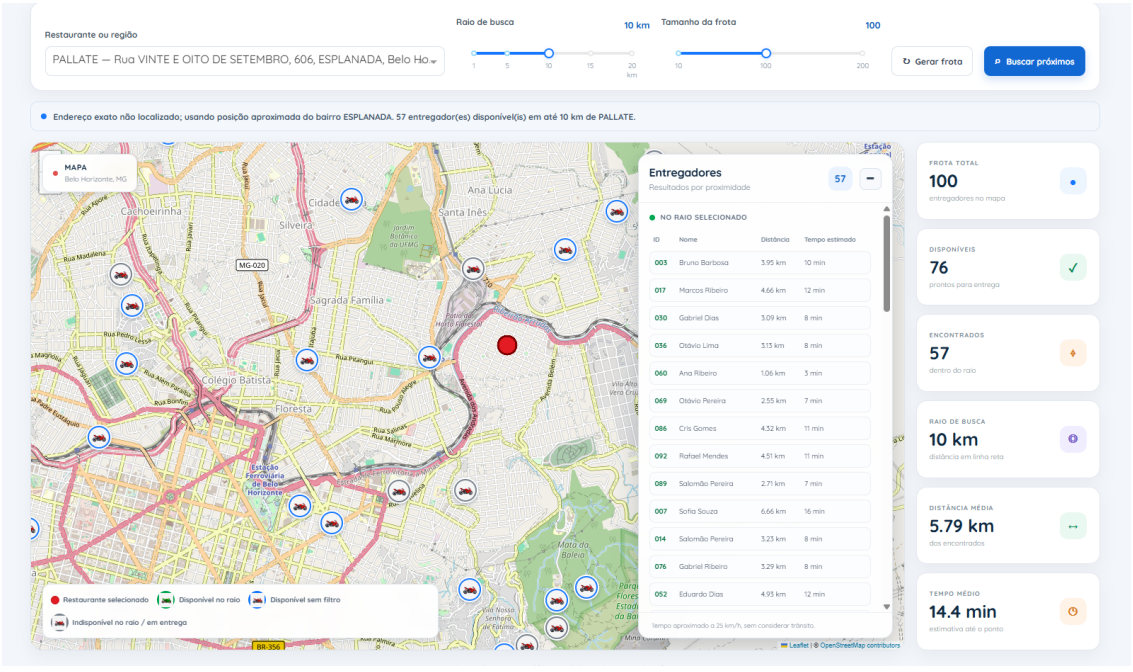


Figura 3: Resultado do Teste 1: Interface exibindo a geolocalização e os entregadores encontrados no raio de 10 km.

6.1.2 Teste 2: Reno Delikatessen ltda

Segundo cenário de validação, foram definidos os seguintes parâmetros de execução:

- **Estabelecimento de Referência:** Reno Delikatessen.
- **Raio de Busca:** 5km.
- **Tamanho da Frota:** 50 entregadores gerados aleatoriamente

A Figura 5 demonstra o resultado parcial da geração da frota e da filtragem espacial sob esses parâmetros:

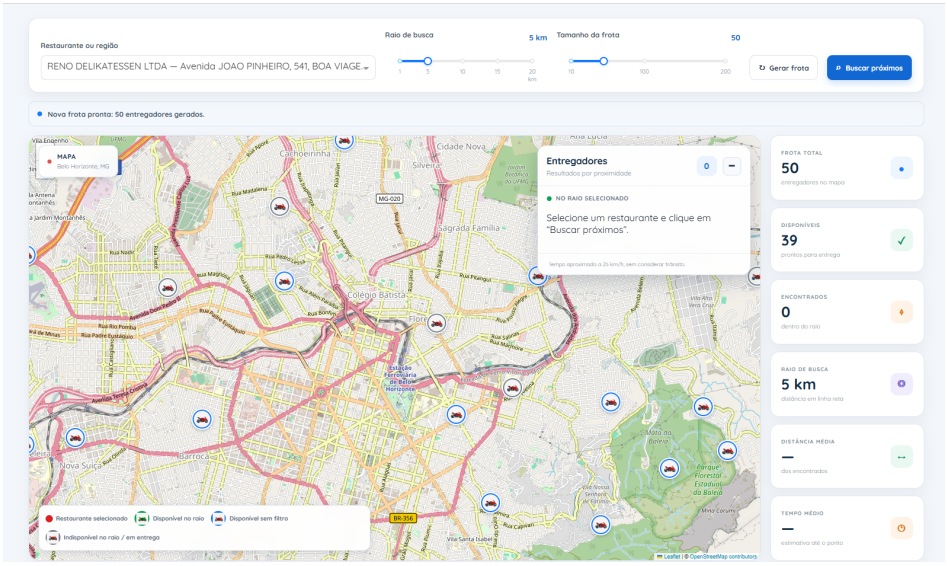


Figura 4: Resultado do Teste 2: Visualização da frota e busca para o Rino.

A Figura 5 demonstra o resultado da execução com os parâmetros acima, destacando a visualização cartográfica no mapa interativo, a listagem de entregadores disponíveis por proximidade no painel lateral e as métricas gerais da frota.

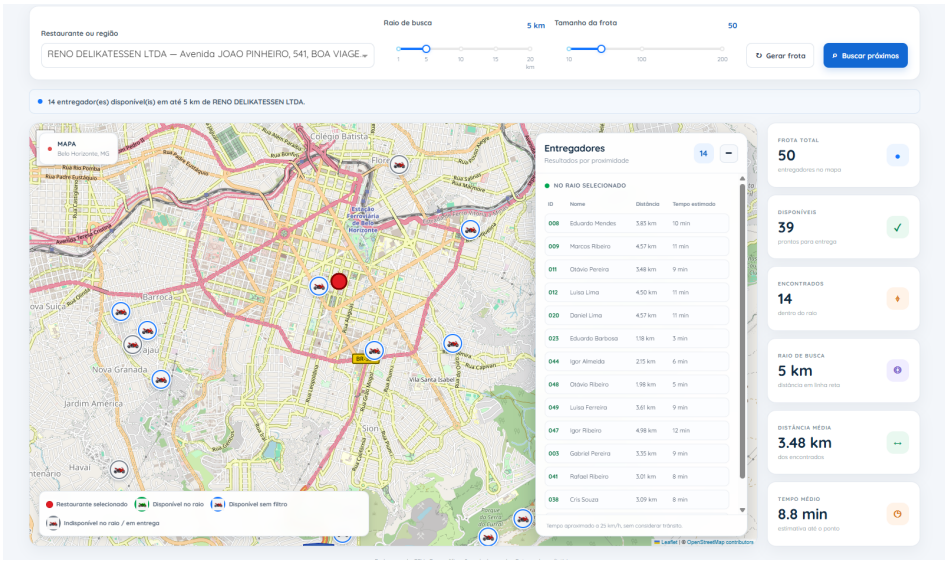


Figura 5: Resultado do Teste 2: Interface exibindo a geolocalização e os entregadores encontrados no raio de 50 km.

7 Conclusão

O trabalho prático permitiu consolidar a aplicação de conceitos avançados de Geometria Computacional, estruturação de dados e desenvolvimento. A combinação entre o Geohash e a Radix Tree permitiu reduzir de forma eficiente o espaço de busca, otimizando o cálculo de distâncias. Foi possível validar a aplicabilidade da ferramenta em cenários reais, mesmo que simulados, de rastreamento e logística urbana.